LECTURE 3

# MCP, Tool calling, Agent orchestration

28 October 2026

LLMOps with Databricks



LECTURE 3

# Agenda

01 **Agents & orchestration**

What is an agent? Workflows, tool control and agentic RAG.

02 **Agent tools & MCP**

Custom tools, Model Context Protocol and Databricks MCP.

03 **Genie agent as a tool**

Give an agent access to structured data through Genie.

04 **Lakebase for agent memory**

Persist conversation history and read it back on the next turn.



# Agents & orchestration

# What is an agent?

An agent is an LLM configured with instructions and tools.

## Instructions

The system prompt defines the agent's role and how it should respond.

## Model

The language model and settings used to generate responses and tool requests.

## Tools

Functions or APIs the model can request to retrieve information or perform actions.

## Optional runtime behavior

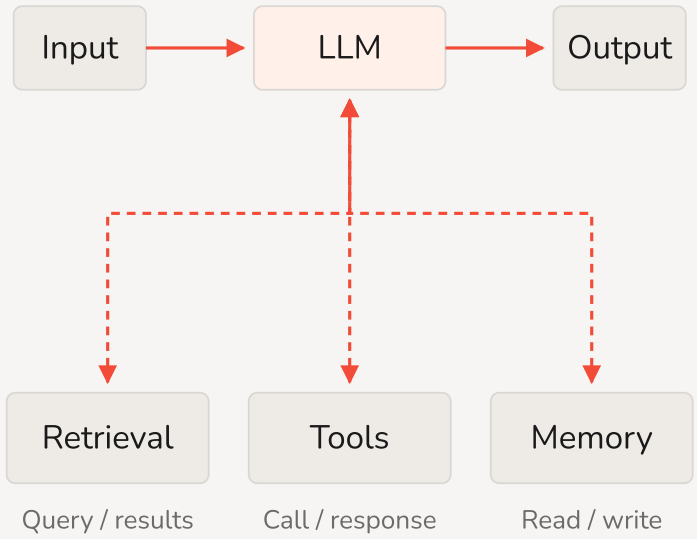
Handoffs, guardrails and structured outputs extend the agent configuration.

Source: OpenAI Agents SDK: Agents

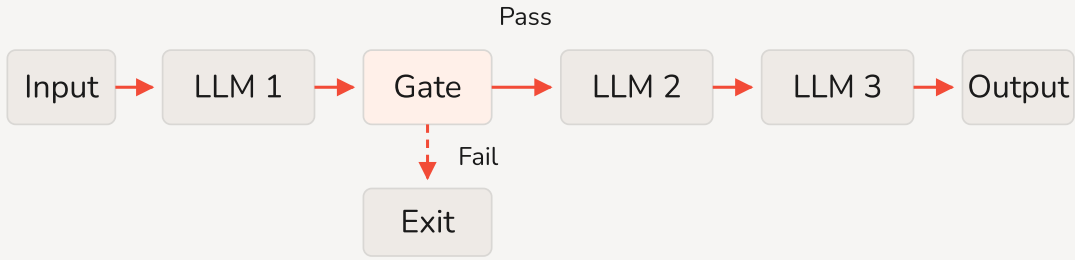


# Agents as augmented LLMs

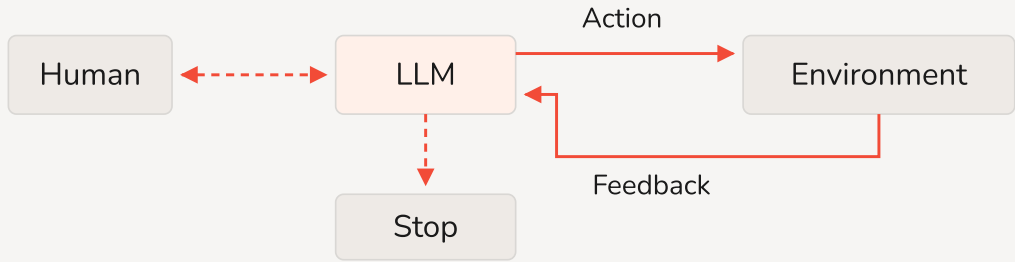
## An augmented LLM




## A predefined workflow

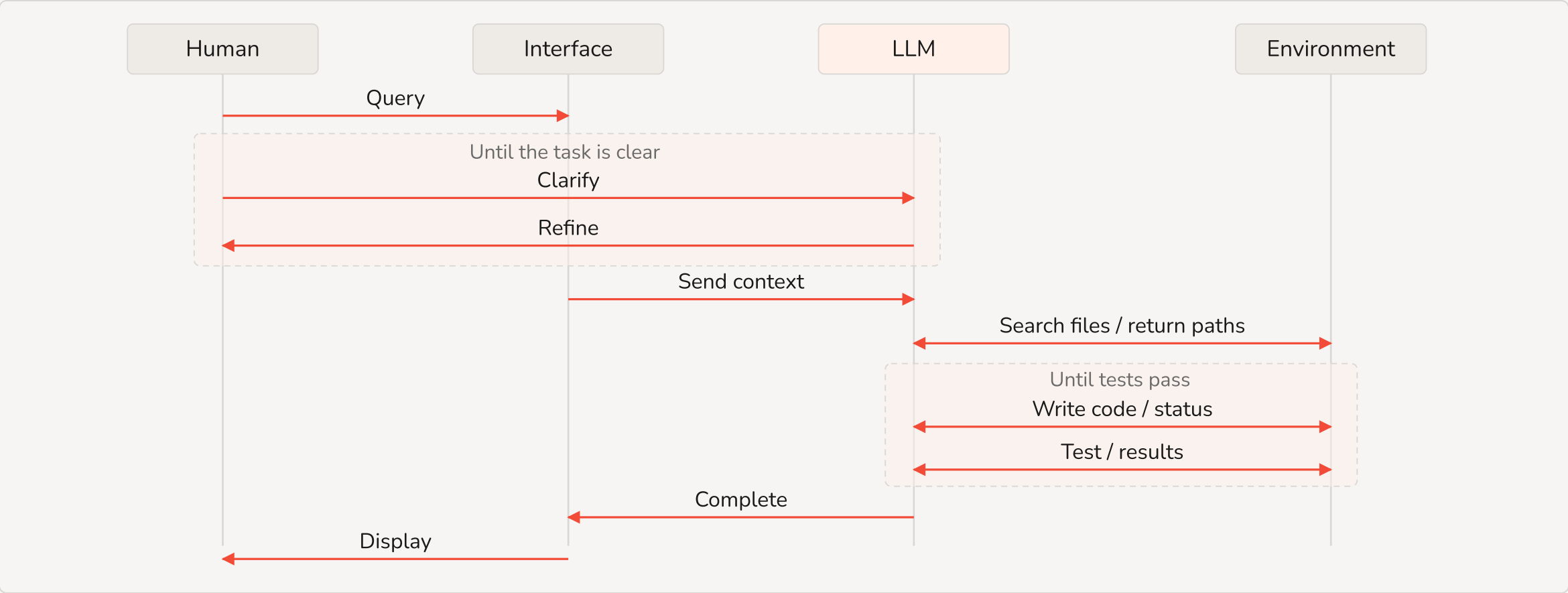



## An agent




Source: [Anthropic: Building effective agents]

# The agent execution loop




Source: Anthropic: Building effective agents

ANTHROPIC'S DISTINCTION (PARAPHRASED)

# LLM workflows versus agents

The distinction is who directs the process and tool use.

## LLM workflows

- Code defines the paths through the model and tools.
- Developers control the order and branching.
- Useful for repeatable, well-defined tasks.

**The LLM follows a predefined process.**

## Agents

- The model decides which step to take next.
- It selects tools and adapts to their results.
- Useful when the task needs flexible decisions.

**The LLM directs the process as it runs.**

> **Workflows: control in code. Agents: control in the model.**

Source: Anthropic: Building effective agents



OUR COURSE DEFINITION

# Agent = LLM with tool control

Following the OpenAI Agents SDK model of an agent.

An agent has:

- An LLM with instructions
- Access to tools, such as APIs or functions
- The ability to **decide when and how to use them**

**The decision to use the tool is the key.**

The LLM chooses the tool call: agent.

Application code predetermines the tool call: workflow.

The application executes tool requests in both cases. This is our course distinction, not a universal definition.



# Simple RAG versus agentic RAG

## Simple RAG

User question → application retrieves chunks → application builds a prompt → model answers.
Retrieval is fixed and happens outside the model.

## RAG as a tool

User question → model decides to call `search_papers()` → tool returns chunks → model answers.
The model controls when and how retrieval happens.

**Lecture 2 built the retrieval layer. Lecture 3 gives the model control over it.**

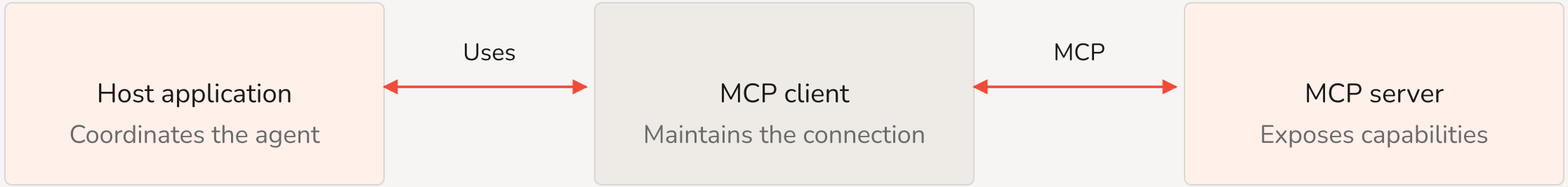


# Agent tools & MCP



# MCP core concepts

A standard connection between an AI application and external capabilities.




## Tools

Callable functions that retrieve information or perform actions.

`search_papers(query)`

## Resources

Content the application can read and supply as context.

`Document or file contents`

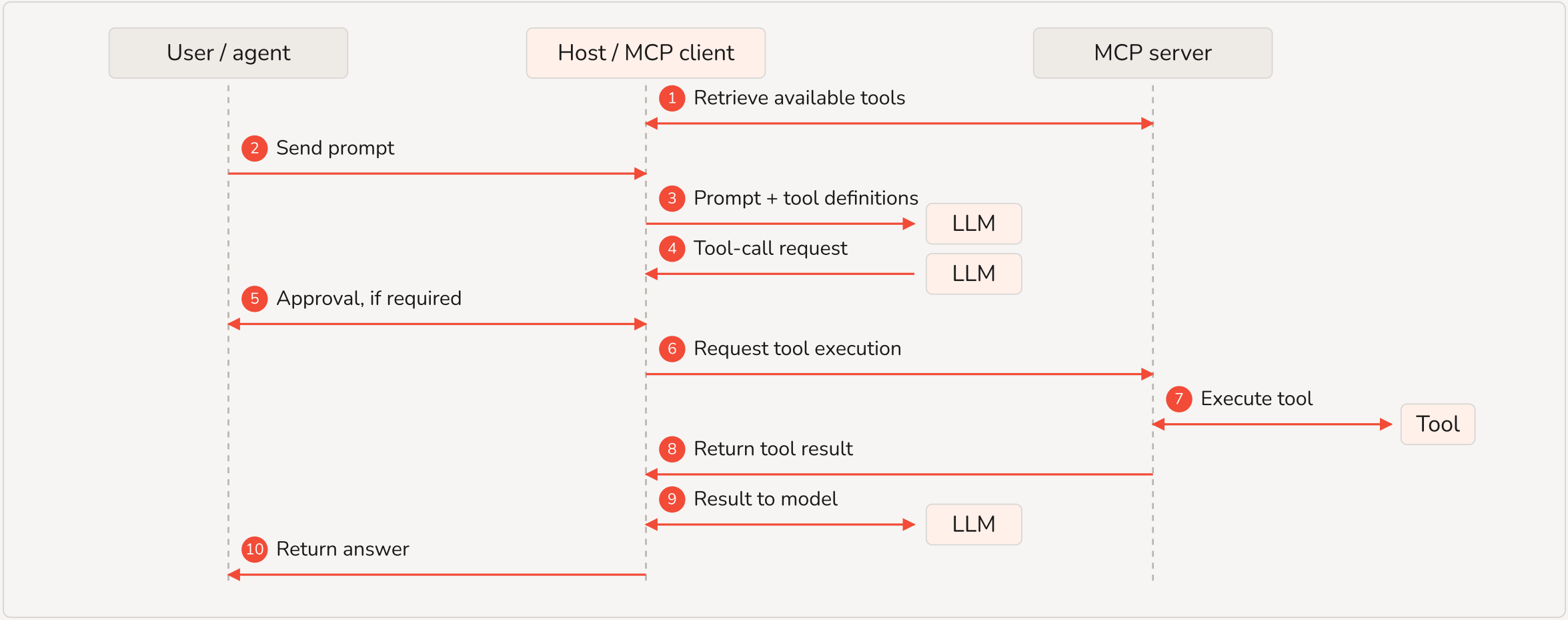
## Prompts

Reusable prompt templates exposed by the server.

`Research summary template`

> The host calls the model. The MCP client handles communication with the server.

# MCP tool calls




Adapted from Elastic Security Labs: MCP tool calls. Approval depends on the host's policy.

IMPLEMENTATION

# Tool-calling loop over pgvector

**1. User**

Asks a research question.

→

**2. LLM**

Chooses `search_papers`.

→

**3. Tool**

Runs pgvector hybrid search.

→

**4. LLM**

Uses results and cites evidence.

```
search_papers(query, num_results, year_filter)
→ LakebasePgVectorManager.hybrid_search(...)
→ chunks, metadata, scores
→ grounded response
```



# MCP versus custom functions

| Question | MCP server | Custom function |
| --- | --- | --- |
| **Setup** | Connect to an existing server. | Write and register Python code. |
| **Reuse** | One server can support many agents. | Usually designed for one application. |
| **Security** | Authentication and authorization can be managed centrally. | The application owns the security implementation. |
| **Customization** | Use the capabilities the server exposes. | Implement custom business logic directly. |

**Use MCP for standard operations.**

Use custom functions when the application needs specialized logic.



# Managed MCP servers

Databricks provides managed MCP endpoints for common platform capabilities.

| Server | Purpose | Current route |
| --- | --- | --- |
| **AI Search** | Query a Databricks AI Search index. | /api/2.0/mcp/ai-search/{catalog}/{schema}/{index} |
| **Genie** | Ask natural-language questions over a Genie Space. | /api/2.0/mcp/genie/{space_id} |
| **Unity Catalog functions** | Invoke registered functions. | /api/2.0/mcp/functions/{catalog}/{schema} |
| **DBSQL** | Run permitted SQL operations. | /api/2.0/mcp/sql |

The legacy `vector-search` route may remain available for compatibility; new material should use AI Search terminology.



# Unity Gateway

Govern and access your AI. Learn more

What's new | Govern

Models | Providers | MCPs | Skills | Agents

## Featured MCPs

**Connect your coding agents**
Use your MCP servers in your coding agents, governed by Unity Gateway.

**Genie One**
Ask natural-language questions against your data

**Atlassian**
Search Jira issues and Confluence pages

**GitHub**
Search code, manage issues and pull requests

**Slack**
Send and search messages across channels

Search Servers | Type: All | Owned by me | + MCP

| Name | Description | Governance | Status | Location | Created By |
|---|---|---|---|---|---|
| Bakehouse Sales Starter Space | Welcome to AI/BI Genie, powered ... | | Active | | maria@cauchy.io |
| Academic Research Papers | | | Active | | basaktugceeskili@gmail.c |
| NYC Taxi Trip Analysis | | | Active | | victor@cauchy.io |
| arxiv-curator-space | | | Active | | basak@marvelousmlops.ic |
| llmops_dev.arxiv | | | Active | llmops_dev.arxiv | basaktugceeskili@gmail.c |
| course_ops.registry | | | Active | course_ops.registry | sp-onboarding-llmops |

# Why use MCP?

## Standardized

Agents use a common protocol across different systems.

## Reusable

One server can expose a capability to many agents.

## Managed

The platform can operate the server and its connections.

## Enterprise ready

Authentication, authorization, and scaling are part of the platform design.



# UC functions as governed MCP tools

## Define once

```
CREATE FUNCTION
llmops_dev.arxiv.add_numbers(a
DOUBLE, b DOUBLE) RETURNS DOUBLE
LANGUAGE PYTHON RETURN a + b;
```

## Expose through MCP

```
server =
"/api/2.0/mcp/functions/llmops_dev/ar
xiv"; client discovers the function;
model can call it
```

**Why MCP?**

UC governance, permissions, lineage, and reuse stay in place while agents get a standard discovery and tool-calling interface.

Use direct SQL when the workflow is fixed. Use UC Functions MCP when multiple agents need governed, discoverable functions.



# Genie agent as a tool



THE GENIE FAMILY

# Three Genie experiences



## Genie Code

**For technical users**

Writes code and works within the lakehouse.

Workspace + terminal
**Genie Code CLI (Beta)**



## Genie One

**For business users**

Explore data, dashboards and apps in one place.

Previously
**Databricks One**



## Genie Agents

**For business & technical users**

Answer domain-specific questions over curated data.

Previously **Genie Spaces**

**OUR FOCUS: AGENT AS A TOOL**

Genie One · Genie Code CLI · Genie Agents

GENIE AGENT

# A curated agent for the paper collection

Ask a data question and receive SQL, result tables and visualizations.

## Data and compute

Unity Catalog table:
`{catalog}.{schema}.arxiv_papers`
SQL warehouse:
`wh-ops-llmops`

## Domain knowledge

Configure instructions, column descriptions and example SQL.
Explain terms such as “recent papers” and which publication date to use.

**Example: “Find the last 10 papers published.”**

Follow up: “Return the authors of those papers.”

Course setup: notebook 3.2b. APIs and configuration still use names such as `space_id` and `genie_space_id`.
Databricks: Genie Agents



AGENT ORCHESTRATION

# Genie as a tool in our agent

The outer agent chooses which capability fits the question.

### Course agent

User question + instructions

The model selects a tool and uses its result to answer.

### Genie Agent through MCP

Structured questions over the curated paper table.

**“How many papers were published each month?”**

### Paper search through pgvector

Retrieve relevant passages from parsed papers.

**“What do these papers say about attention?”**

> Genie handles the data question. The course agent combines the result with the rest of the conversation.

Databricks: Genie Agent MCP · Course implementation: notebook 3.2.



COURSE IMPLEMENTATION

# Connect the Genie Agent through MCP

```
from databricks.sdk import WorkspaceClient
from arxiv_curator.mcp import create_mcp_tools
import asyncio

w = WorkspaceClient()
genie_url = (
    f"{w.config.host}/api/2.0/mcp/genie/"
    f"{cfg.genie_space_id}"
)
genie_tools = asyncio.run(create_mcp_tools(w, [genie_url]))

# One agent, two capabilities
tools = [search_papers_tool, *genie_tools]
```

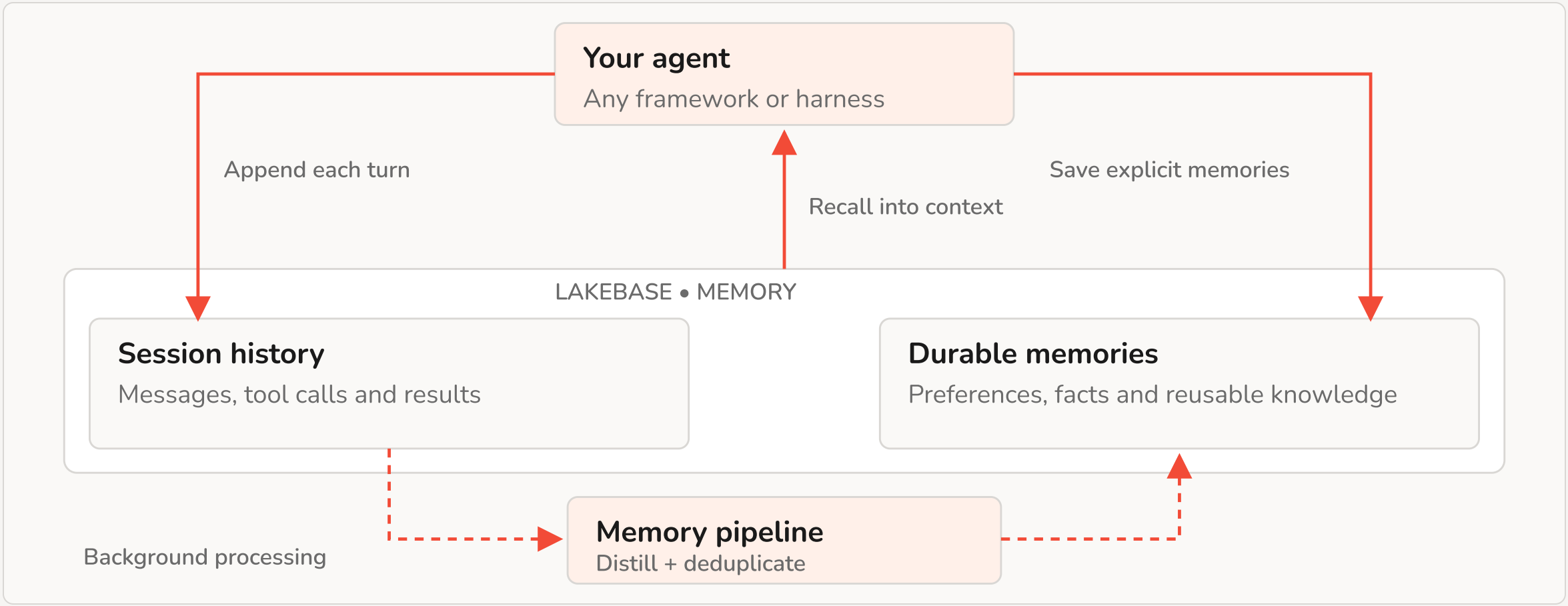
`create_mcp_tools` is our course helper: it discovers tool schemas and creates callables for the agent loop.

Notebook 3.2 provides `cfg` and `search_papers_tool`. The Genie Agent and its underlying data must be accessible to the calling identity.



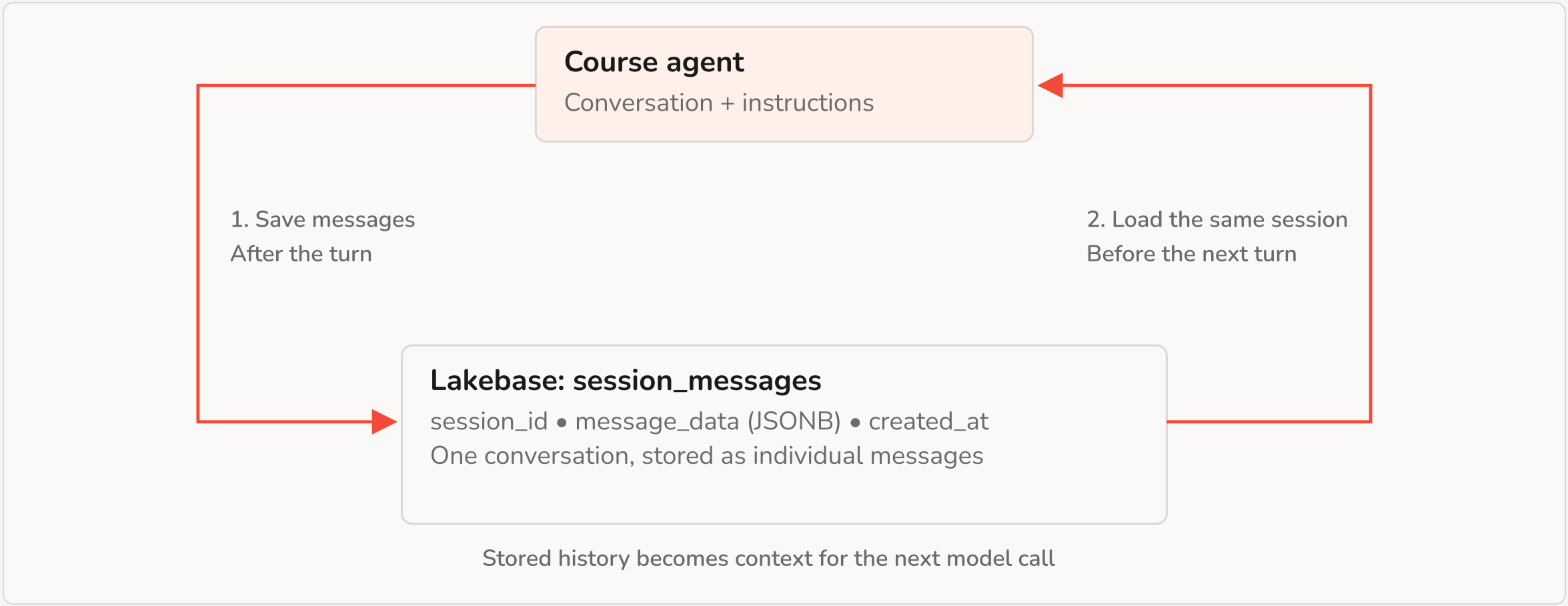
# Lakebase for agent memory

# From conversation history to durable memory




Conceptual architecture adapted from the supplied diagram. Distillation extracts useful knowledge from history; deduplication consolidates repeated memories.

# Keep it simple: save, then read back




**No distillation in this example.** We read the stored messages back by session ID. Notebook 3.3: `save_messages()` and `load_messages()`.

# Homework



HOMEWORK

# Build an agent with tools and memory

01 Review the agent and MCP materials; identify tools for your use case.

02 Sketch the orchestration: code-controlled steps and model decisions.

03 Connect tools through custom functions or MCP, including Genie.

04 Save conversation history in Lakebase and load it on the next turn.

05 Submit your implementation and design choices through a pull request.



# Questions?

Next lecture: MLflow for GenAI

